

LAPORAN

Pemrograman Citra Digital



Nama : Mohamad Irsyad Ibrahim Faqih
NIM : 202431179
Kelas : F
Nama Dosen : Rizqia Cahyaningtyas, ST., M.Kom
No. Bangku :
Nama Asisten :
1. Erin Katholica Sakraly Putri Marrison
2. Morgan Immanuel Nainggolan

LABORATORIUM COMPUTER NETWORK
INSTITUT TEKNOLOGI PLN
2026

DAFTAR ISI

DAFTAR ISI	2
BAB I	3
PENDAHULUAN	3
1.1 Rumusan Masalah	3
1.2 Tujuan Masalah	3
1.3 Manfaat Masalah	3
BAB II	4
LANDASAN TEORI	4
BAB III	7
PEMBAHASAN	7
BAB IV	16
PENUTUP	16
DAFTAR PUSTAKA	17

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

- Bagaimana cara mengimpor citra asli (`EA_GAMES.png`) serta mengubahnya ke format skala keabuan (grayscale) menggunakan OpenCV?
- Bagaimana menerapkan operasi konvolusi High-Pass Filter menggunakan kernel Laplacian untuk menonjolkan detail tepi objek pada citra?
- Bagaimana alur logika perulangan manual (nested for-loop) dalam membangun filter spasial rata-rata lokal dan Filter Batas berbasis ketetanggaan piksel?
- Bagaimana perbandingan efisiensi performa antara Average Filter dan Median Filter saat digunakan untuk mereduksi variasi derau (Salt & Pepper, Gaussian, dan Speckle)?

1.2 Tujuan Masalah

- Mampu memahami langkah dasar pembacaan gambar, konversi ruang warna BGR menjadi skala keabuan, serta analisis dimensi matriksnya.
- Mampu mengeksekusi operasi konvolusi penajaman tepi (High-Pass Filtering) menggunakan matriks kernel secara presisi.
- Mampu merancang program perulangan mandiri untuk memanipulasi nilai piksel pusat berdasarkan kondisi intensitas piksel tetangga di sekelilingnya.
- Mampu menganalisis hasil restorasi citra untuk menentukan jenis filter spasial yang paling efektif dan adaptif terhadap karakteristik derau tertentu.

1.3 Manfaat Masalah

- Secara Teoretis memperdalam pemahaman mengenai konsep matriks dua dimensi, prinsip kerja konvolusi, dan teori ketetanggaan piksel lokal pada citra digital.
- Secara Praktis menguasai fungsi-fungsi penting dari pustaka OpenCV, NumPy, dan Matplotlib dalam melakukan manipulasi serta visualisasi data citra.
- Kemampuan Restorasi memiliki kemampuan teknis untuk menganalisis tingkat kerusakan citra akibat derau (noise) serta solusi penapisannya.

BAB II

LANDASAN TEORI

Konsep Dasar dari Operasi Konvolusi dalam Pengolahan Citra Digital

Kalau kita bicara soal pengolahan citra digital, operasi konvolusi itu bisa dibilang sebagai salah satu menu wajib atau fondasi paling krusial. Secara sederhana, konvolusi adalah proses matematis di mana kita mengalikan dua buah fungsi untuk menghasilkan fungsi baru yang merepresentasikan perubahan dari citra tersebut. Dalam penerapannya, kedua fungsi ini berupa matriks citra asli yang berukuran besar dan sebuah matriks kecil ganjil yang biasa kita sebut sebagai kernel atau filter. Komputer akan menggeser kernel ini piksel demi piksel ke seluruh area gambar untuk menghitung nilai intensitas yang baru.

Proses berjalannya operasi konvolusi ini mengikuti beberapa tahapan berulang yang logis. Kernel pertama-tama diletakkan di sudut kiri atas matriks citra, tepat di atas piksel target yang berada di tengah-tengah jangkauan kernel tersebut. Setelah itu, setiap nilai piksel citra yang tertutup oleh kernel akan dikalikan secara langsung dengan nilai bobot kernel yang bersesuaian di posisi yang sama. Semua hasil perkalian elemen tersebut kemudian dijumlahkan untuk menghasilkan satu nilai tunggal yang menjadi nilai intensitas baru bagi piksel target di matriks citra keluaran. Kernel kemudian bergeser satu piksel ke kanan atau ke bawah dan mengulangi proses yang sama hingga seluruh area citra selesai dipindai.

Tujuan utama dari operasi konvolusi ini sebenarnya sangat fleksibel dan bergantung pada nilai-nilai bobot yang kita tanam di dalam kernel itu sendiri. Jika nilai kernel diatur sedemikian rupa, konvolusi bisa berfungsi untuk menghaluskan gambar, menajamkan tepi objek, hingga mendeteksi pola-pola tertentu yang menjadi modal awal dalam algoritma kecerdasan buatan.

Metode Ketetanggaan dalam Pengolahan Citra

Ketika komputer membaca sebuah citra, komputer tidak melihat gambar itu secara utuh langsung seperti mata manusia, melainkan piksel demi piksel. Nah, dalam memproses sebuah piksel target, nilai piksel itu sering kali sangat dipengaruhi atau memiliki keterikatan spasial dengan piksel-piksel yang berada di sekitarnya. Konsep inilah yang kita sebut dengan Metode Ketetanggaan Piksel. Metode ketetanggaan menetapkan aturan mengenai piksel mana saja yang dianggap bertetangga langsung dengan sebuah piksel pusat pada koordinat tertentu.

Berdasarkan struktur spasial dua dimensi, ada dua jenis konektivitas ketetanggaan standar yang paling sering dipakai dalam pengolahan citra:

- a. 4-Ketetanggaan : Pada metode ini, piksel yang dianggap tetangga resmi dari piksel pusat hanyalah piksel-piksel yang menyentuh sisi horizontal dan vertikalnya secara langsung pada arah atas, bawah, kiri, dan kanan, sehingga arah diagonal tidak dihitung.

- b. 8-Ketetanggaan : Metode ini jauh lebih inklusif karena selain mengambil empat piksel yang berada di sisi horizontal dan vertikal, ia juga memasukkan empat piksel yang menyentuh sudut diagonal piksel pusat secara langsung.

Pemahaman mengenai metode ketetanggaan ini sangat krusial ketika kita mengimplementasikan algoritma untuk menghitung jumlah objek, operasi morfologi seperti erosi dan dilasi, hingga proses pelacakan batas kontur objek. Jika salah menentukan jenis ketetanggaan, komputer bisa salah menginterpretasikan objek. Misalnya, dua objek yang aslinya terhubung secara diagonal bisa dianggap terpisah sama sekali oleh komputer jika kita hanya mengaktifkan aturan 4-ketetanggaan.

Prinsip Kerja Average Filter dan Median Filter dalam Mengurangi Noise beserta Kelebihan dan Kekurangannya

Derau atau noise sering kali menyusup ke dalam citra akibat gangguan sensor kamera atau distorsi saat proses transmisi data. Untuk mengatasinya, kita memerlukan filter spasial seperti Average Filter dan Median Filter. Meski tujuannya sama-sama mereduksi noise, cara kerja dan hasil akhir keduanya sangatlah bertolak belakang karena yang satu bekerja secara linear berbasis rata-rata matematika, sedangkan yang satu bekerja secara non-linear berbasis pengurutan nilai.

Average Filter bekerja dengan memanfaatkan operasi konvolusi di mana seluruh nilai elemen di dalam kernelnya dibuat seragam. Filter ini mengambil semua nilai piksel dalam jangkauan kernel, menjumlahkan seluruh nilai intensitas piksel tersebut, lalu membaginya dengan jumlah total piksel yang ada di dalam lingkup kernel tersebut. Nilai rata-rata inilah yang menggantikan piksel pusat. Kelebihan utamanya adalah sangat efektif dan cepat untuk mengurangi noise yang tersebar merata dengan fluktuasi halus seperti Gaussian noise. Namun, kekurangannya adalah filter ini mengaburkan tepi-tepi tajam pada objek sehingga detail penting pada citra bisa hilang dan gambar terlihat blur.

Di sisi lain, Median Filter adalah filter non-linear yang bekerja dengan cara mengambil seluruh nilai piksel tetangga di dalam lingkup kernel, lalu mengurutkan nilai-nilai intensitas piksel tersebut dari yang terkecil hingga terbesar. Setelah terurut, nilai yang berada tepat di posisi tengah atau median akan diambil untuk menggantikan nilai piksel pusat. Kelebihan terbesarnya adalah sangat digdaya dalam melenyapkan Salt & Pepper noise yang ekstrem tanpa mengorbankan ketajaman tepi objek. Sedangkan kekurangannya adalah proses komputasinya lebih intensif karena komputer harus melakukan fungsi sorting nilai piksel di setiap pergeseran kernel, serta risiko hilangnya objek-objek berukuran sangat kecil jika ukuran kernel terlalu besar.

Speckle Noise dan Bagaimana Ia Berbeda dari Gaussian Noise

Dalam dunia pemrosesan gambar, mengenali jenis noise itu ibarat mendiagnosis penyakit sebelum memberikan filter yang tepat. Dua jenis noise yang sering muncul namun memiliki karakteristik fisik dan matematis yang sangat bertolak belakang adalah Speckle noise dan Gaussian noise. Perbedaan fundamental di antara keduanya terletak pada cara gangguan tersebut berinteraksi dengan nilai intensitas asli dari piksel citra yang dikenainya.

Speckle noise adalah jenis derau yang bersifat multiplikatif, yang berarti intensitas derau ini bergantung langsung pada nilai intensitas piksel citra aslinya dengan model matematis hasil perkalian antara citra asli dan komponen derau. Derau ini umumnya dijumpai pada sistem pencitraan aktif yang menggunakan gelombang koheren seperti pada citra radar atau citra medis ultrasonografi (USG). Secara visual, speckle noise terlihat seperti tekstur granular berpola bintik-bintik acak yang mengelompok dan mengganggu ketajaman detail visual organ atau wilayah geografis yang dipindai.

Sebaliknya, Gaussian noise adalah derau yang bersifat aditif, di mana nilai gangguan ini ditambahkan secara langsung ke nilai piksel asli tanpa memedulikan seberapa besar atau kecil intensitas piksel tersebut. Sesuai namanya, distribusi probabilitas dari nilai derau ini mengikuti kurva normal atau distribusi Gaussian yang biasanya disebabkan oleh masalah teknis perangkat keras seperti suhu panas pada sensor kamera atau interferensi saluran transmisi elektronik. Secara visual, Gaussian noise memunculkan butiran-butiran halus yang tersebar merata di seluruh permukaan citra sehingga membuat gambar terlihat seperti diselimuti kabut halus atau semut-semut TV tipis.

Manfaat Operasi Konvolusi dalam Bidang Komputer Visi

Komputer visi adalah cabang ilmu yang berupaya membuat komputer mampu melihat dan memahami makna dari sebuah citra atau video layaknya manusia. Dalam alur kerjanya, operasi konvolusi bertindak sebagai gerbang utama sekaligus mesin penggerak paling vital karena tanpa konvolusi, komputer vision akan kesulitan mengekstrak informasi bermakna dari tumpukan data piksel mentah yang begitu masif.

Ada beberapa manfaat konkret dari penerapan operasi konvolusi ini di ranah komputer visi:

- a. Ekstraksi Fitur dan Deteksi Tepi: Konvolusi memungkinkan komputer mendeteksi komponen primitif dari sebuah gambar seperti garis, sudut, kontur, hingga perubahan kontras intensitas tajam pada tepi objek melalui kernel khusus seperti operator Sobel atau Canny.
- b. Tulang Punggung Modern Deep Learning: Operasi konvolusi menjadi dasar utama dari arsitektur Convolutional Neural Network (CNN) yang mendominasi aplikasi komputer visi terkini seperti sistem pengenalan wajah, segmentasi citra medis, hingga navigasi pada mobil otonom.

BAB III

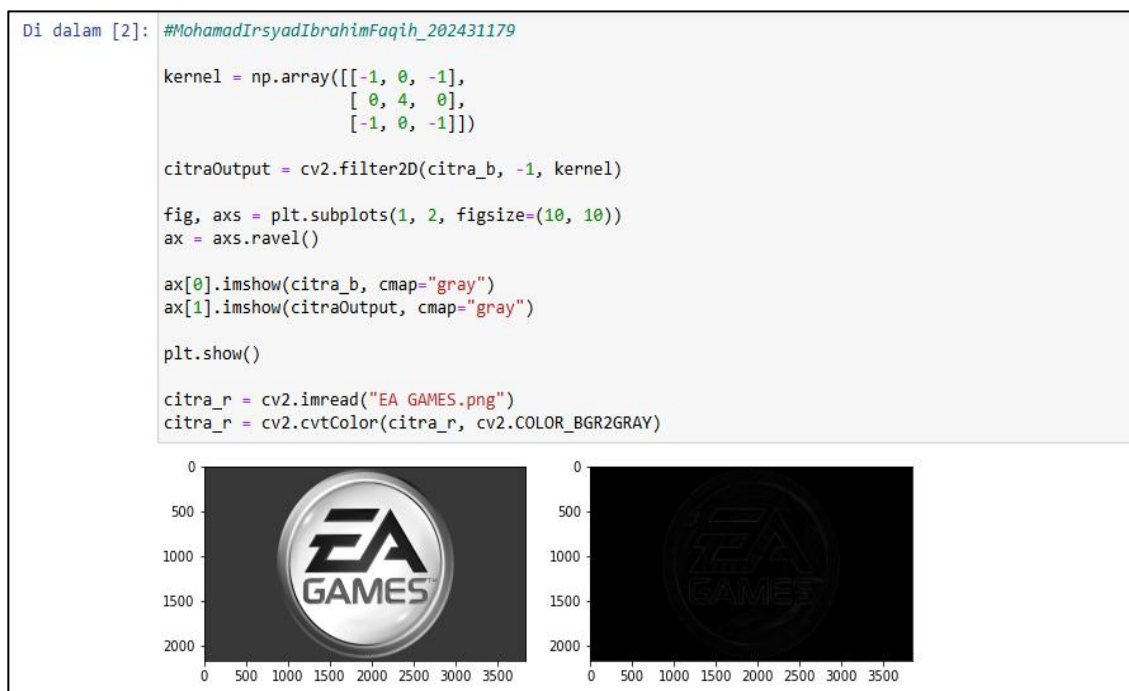
PEMBAHASAN

Konvolusi



Untuk bagian tahap awal, prosesnya dimulai dengan mengimpor library OpenCV, NumPy, dan Matplotlib, lalu membaca input gambar EA_GAMES.png.

Gambar tersebut kemudian diubah dari ruang warna BGR menjadi grayscale memakai perintah `cv2.cvtColor`. Setelah itu, kita lakukan print pada dimensi matriksnya untuk mengecek ukuran piksel gambarnya, dan langkah ini diakhiri dengan menampilkan visualisasi citra abu-abu tersebut ke layar plot menggunakan Matplotlib.



Pada tahap ini, proses pengolahan citra difokuskan pada penerapan penyaringan dua dimensi menggunakan matriks kustom berukuran 3x3 yang didefinisikan lewat variabel kernel melalui perintah `np.array` untuk membentuk filter pendeteksi tepi. Matriks tersebut kemudian diaplikasikan ke seluruh piksel gambar skala abu-abu menggunakan fungsi `cv2.filter2D` dengan parameter kedalaman warna -1 agar struktur datanya tetap sama, lalu hasilnya disimpan ke dalam variabel `citraOutput`. Untuk menyajikan perbandingannya, kanvas visualisasi disiapkan menggunakan modul `plt.subplots` dari Matplotlib dengan susunan satu baris dua kolom yang diratakan lewat fungsi `axs.ravel()`, sehingga gambar asli diletakkan pada indeks kiri `ax[0]` dan hasil penyaringan diposisikan pada indeks kanan `ax[1]` menggunakan peta warna abu-abu (`cmap="gray"`) sebelum akhirnya ditampilkan ke layar secara bersamaan lewat perintah `plt.show()`. Sebagai langkah penutupan, sistem melakukan pembacaan ulang terhadap file "EA GAMES.png" ke dalam variabel baru bernama `citra_r` yang langsung dikonversi menjadi skala abu-abu memakai perintah `cv2.cvtColor` guna menyiapkan data bersih yang terisolasi untuk proses analisis berikutnya tanpa memengaruhi variabel yang telah digunakan.

Di dalam [3]: `#MohamadIrsyadIbrahimFaqih_202431179`

```
citra_r = cv2.imread("EA GAMES.png")
citra_r = cv2.cvtColor(citra_r, cv2.COLOR_BGR2GRAY)

plt.imshow(citra_r, cmap="gray")
plt.show()
```



Pada tahap pengolahan citra digital ini, pada pembacaan ulang dan visualisasi data citra yang bersih ke dalam variabel baru. Langkah pertama dimulai dengan memuat file dokumen visual asli bernama "EA GAMES.png" ke dalam variabel `citra_r` menggunakan perintah `cv2.imread`. Setelah file tersebut berhasil dimuat, ruang warnanya langsung ditransformasikan dari format bawaan BGR menjadi format grayscale atau skala abu-abu dengan memanfaatkan fungsi `cv2.cvtColor` dan parameter `cv2.COLOR_BGR2GRAY`. Rangkaian instruksi pada tahapan ini kemudian diakhiri dengan menyiapkan representasi visual dari data abu-abu tersebut ke dalam sebuah plot lewat perintah `plt.imshow` dengan konfigurasi peta warna kustom `cmap="gray"`, sebelum akhirnya ditampilkan secara utuh ke layar melalui eksekusi fungsi `plt.show()`.


```

Di dalam [5]: #202431179_MohamadIrsyadIbrahimFaqih

from skimage.util import random_noise

noise_img_snp = random_noise(citra_r, mode="s&p", amount=0.2)
noise_img_gaussian = random_noise(citra_r, mode="gaussian", mean=0, var=0.01)
noise_img_speckle = random_noise(citra_r, mode="speckle")

noise_img_snp = np.array(255 * noise_img_snp, dtype='uint8')
noise_img_gaussian = np.array(255 * noise_img_gaussian, dtype='uint8')
noise_img_speckle = np.array(255 * noise_img_speckle, dtype='uint8')

Di dalam [6]: #MohamadIrsyadIbrahimFaqih_202431179

fig, axs = plt.subplots(4, 1, figsize=(10, 40))
ax = axs.ravel()

ax[0].imshow(citra_r, cmap="gray")
ax[0].set_title('Citra Asli')

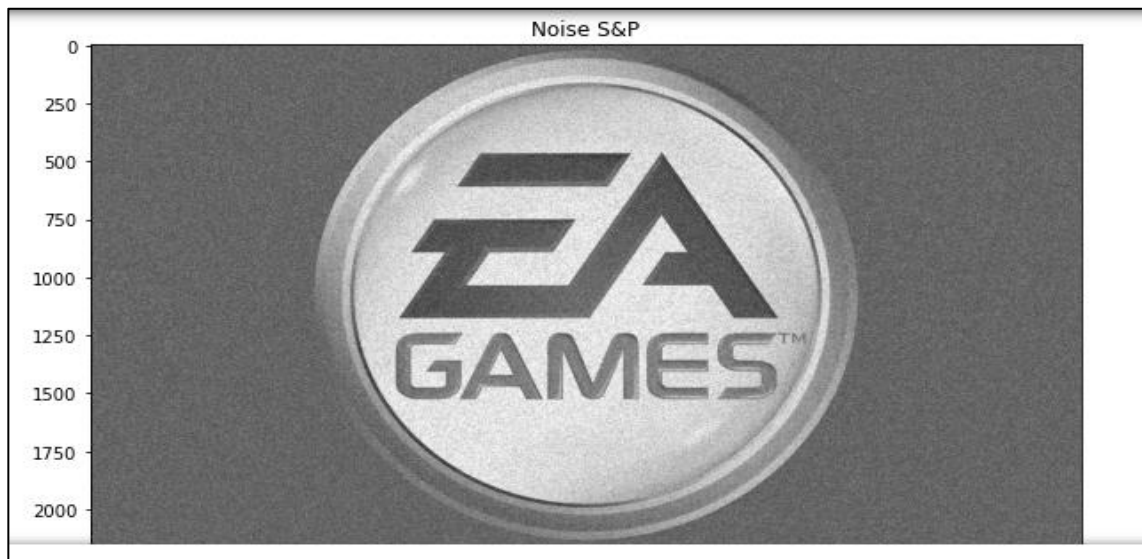
ax[1].imshow(noise_img_snp, cmap="gray")
ax[1].set_title('Noise S&P')

ax[2].imshow(noise_img_gaussian, cmap="gray")
ax[2].set_title('Noise Gaussian')

ax[3].imshow(noise_img_speckle, cmap="gray")
ax[3].set_title('Noise Speckle')

plt.show()

```



Pada tahapan pemrosesan ini, data visual bersih skala abu-abu citra_r diberikan tiga variasi gangguan menggunakan fungsi `random_noise` dari modul `skimage.util` untuk menghasilkan derau Salt and Pepper dengan intensitas 0.2, derau Gaussian bernilai rata-rata nol serta variansi 0.01, dan derau Speckle. Karena pustaka tersebut mengubah intensitas piksel menjadi bentuk desimal, ketiga hasil data berderau tersebut dikalikan dengan nilai 255 dan dikonversi kembali ke tipe data matriks gambar asli ('uint8') lewat fungsi `np.array`. Selanjutnya, sebuah kanvas disusun secara vertikal dalam format 4 baris dan 1 kolom dengan dimensi 10x40 menggunakan `plt.subplots` dan diratakan melalui perintah `axs.ravel()`. Berdasarkan susunan indeksnya, sistem memetakan citra asli tanpa gangguan pada `ax[0]`, citra dengan gangguan Salt and Pepper pada `ax[1]`, citra dengan gangguan Gaussian pada `ax[2]`, serta citra dengan gangguan Speckle pada `ax[3]`. Seluruh visualisasi tersebut diberi judul penanda masing-masing serta diatur menggunakan peta warna abu-abu (`cmap="gray"`) sebelum akhirnya dimunculkan ke layar plot melalui perintah `plt.show()`.

```
Di dalam [7]: #MohamadIrsyadIbrahimFaqih_202431179
kernel1_3_3 = np.ones((3,3), np.float32)/9
```

```
Di dalam [8]: #MohamadIrsyadIbrahimFaqih_202431179

img_snp_avg_filter = cv2.filter2D(noise_img_snp, cv2.CV_8U, kernel1_3_3,
                                   (-1, -1), delta = 0,
                                   borderType = cv2.BORDER_DEFAULT)

img_snp_median_filter = cv2.medianBlur(noise_img_snp, 3)
```

```
Di dalam [9]: #MohamadIrsyadIbrahimFaqih_202431179

img_gaussian_avg_filter = cv2.filter2D(noise_img_gaussian, cv2.CV_8U, kernel1_3_3,
                                         (-1, -1), delta = 0,
                                         borderType = cv2.BORDER_DEFAULT)

img_gaussian_median_filter = cv2.medianBlur(noise_img_gaussian, 3)
```

```
Di dalam [10]: #MohamadIrsyadIbrahimFaqih_202431179

img_speckle_avg_filter = cv2.filter2D(noise_img_speckle, cv2.CV_8U, kernel1_3_3,
                                       (-1, -1), delta = 0,
                                       borderType = cv2.BORDER_DEFAULT)

img_speckle_median_filter = cv2.medianBlur(noise_img_speckle, 3)
```

```
Di dalam [11]: #MohamadIrsyadIbrahimFaqih_202431179

fig, axs = plt.subplots(3, 3, figsize = (15, 15))
ax = axs.ravel()

ax[0].imshow(noise_img_snp, cmap = "gray")
ax[0].set_title('Noise S&P')

ax[1].imshow(img_snp_avg_filter, cmap = "gray")
ax[1].set_title('Average Filter S&P Reduction')

ax[2].imshow(img_snp_median_filter, cmap = "gray")
ax[2].set_title('Median Filter S&P Reduction')

ax[3].imshow(noise_img_gaussian, cmap = "gray")
ax[3].set_title('Noise Gaussian')

ax[4].imshow(img_gaussian_avg_filter, cmap = "gray")
ax[4].set_title('Average Filter gaussian Reduction')

ax[5].imshow(img_gaussian_median_filter, cmap = "gray")
ax[5].set_title('Median Filter gaussian Reduction')

ax[6].imshow(noise_img_speckle, cmap = "gray")
ax[6].set_title('Noise speckle')

ax[7].imshow(img_speckle_avg_filter, cmap = "gray")
ax[7].set_title('Average Filter speckle Reduction')

ax[8].imshow(img_speckle_median_filter, cmap = "gray")
ax[8].set_title('Median Filter speckle Reduction')

plt.show()
```

Di dalam [11]: #MohamadIrsyadIbrahimFaqih_202431179

```
fig, axs = plt.subplots(3, 3, figsize = (15, 15))
ax = axs.ravel()

ax[0].imshow(noise_img_snp, cmap = "gray")
ax[0].set_title('Noise S&P')

ax[1].imshow(img_snp_avg_filter, cmap = "gray")
ax[1].set_title('Average Filter S&P Reduction')

ax[2].imshow(img_snp_median_filter, cmap = "gray")
ax[2].set_title('Median Filter S&P Reduction')

ax[3].imshow(noise_img_gaussian, cmap = "gray")
ax[3].set_title('Noise Gaussian')

ax[4].imshow(img_gaussian_avg_filter, cmap = "gray")
ax[4].set_title('Average Filter gaussian Reduction')

ax[5].imshow(img_gaussian_median_filter, cmap = "gray")
ax[5].set_title('Median Filter gaussian Reduction')

ax[6].imshow(noise_img_speckle, cmap = "gray")
ax[6].set_title('Noise speckle')

ax[7].imshow(img_speckle_avg_filter, cmap = "gray")
ax[7].set_title('Average Filter speckle Reduction')

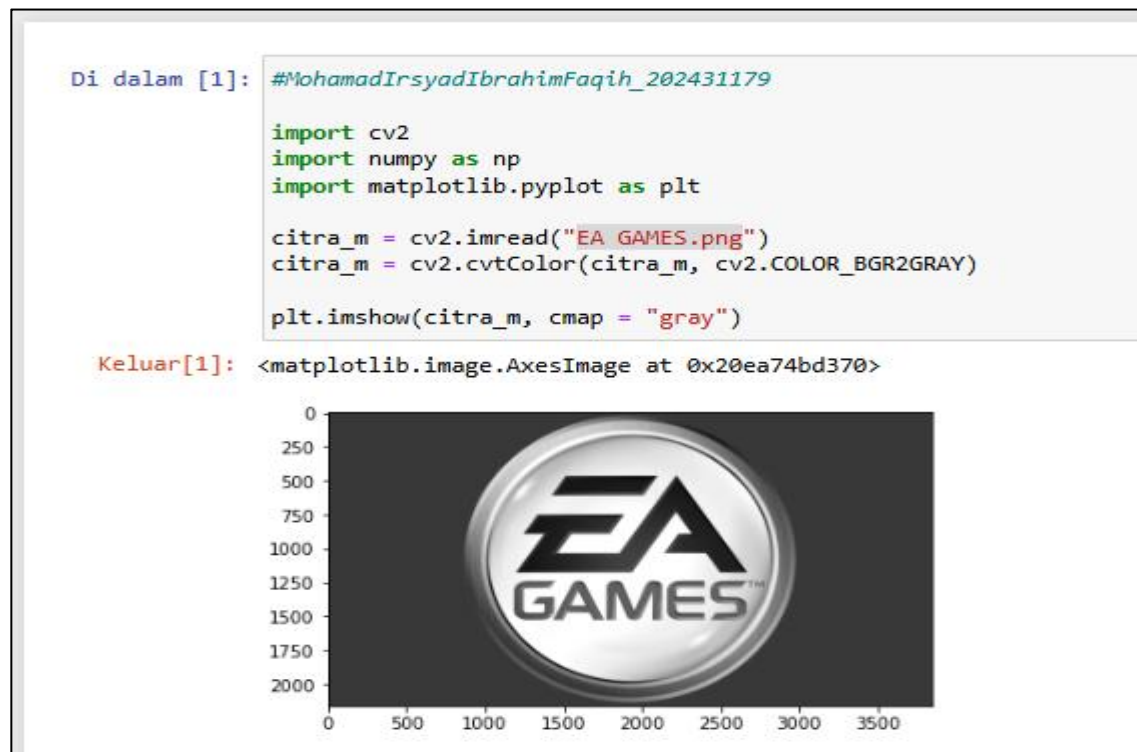
ax[8].imshow(img_speckle_median_filter, cmap = "gray")
ax[8].set_title('Median Filter speckle Reduction')

plt.show()
```

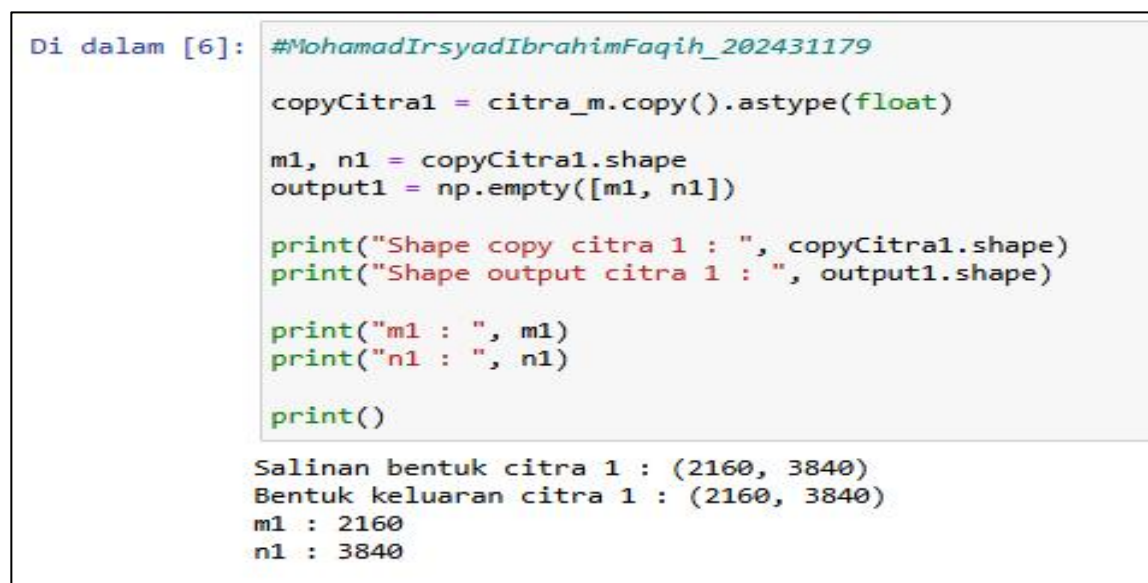


Pada tahapan pemrosesan ini, fokus utama terletak pada reduksi derau (noise reduction) menggunakan dua teknik filtrasi, yaitu Average Filter dan Median Filter, pada tiga jenis citra berderau yang telah dibuat sebelumnya. Langkah awal dimulai dengan mendefinisikan matriks kustom berukuran 3x3 yang berisi nilai satu di setiap elemennya, lalu dibagi dengan 9 lewat perintah `np.ones((3,3), np.float32)/9` untuk membentuk kernel rata-rata yang disimpan dalam variabel `kernel1_3_3`. Selanjutnya, pada masing-masing citra berderau (Salt and Pepper, Gaussian, dan Speckle), diterapkan fungsi `cv2.filter2D` menggunakan kernel tersebut dengan menyertakan tipe data keluaran `cv2.CV_8U` untuk menghasilkan citra terfilter rata-rata (`img_snp_avg_filter`, `img_gaussian_avg_filter`, dan `img_speckle_avg_filter`). Bersamaan dengan itu, operasi filter median juga diaplikasikan pada ketiga citra berderau tersebut menggunakan perintah `cv2.medianBlur` dengan ukuran window sebesar 3 untuk menghasilkan variabel citra terfilter median

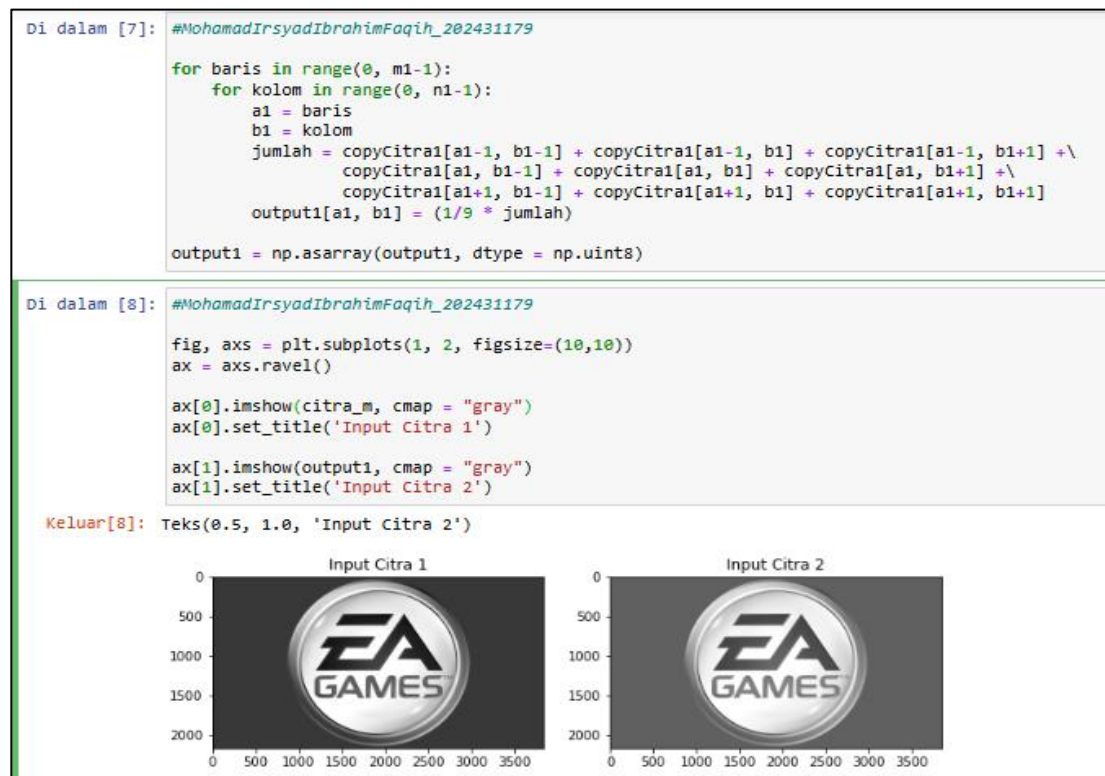
Ketetapan



Pada tahap inisialisasi untuk pemrosesan ketetapan piksel ini, langkah pertama dimulai dengan menyiapkan lingkungan kerja melalui proses impor tiga pustaka utama, yaitu OpenCV, NumPy, dan Matplotlib. Setelah pustaka tersebut siap, sistem membaca file dokumen visual bernama "EA GAMES.png" menggunakan perintah `cv2.imread` dan menyimpannya ke dalam variabel baru bernama `citra_m`. Langkah berikutnya adalah mentransformasikan ruang warna citra tersebut dari format bawaan BGR menjadi format skala abu-abu (grayscale) melalui fungsi `cv2.cvtColor` dengan parameter `cv2.COLOR_BGR2GRAY`. Tahapan awal ini kemudian ditutup dengan menyiapkan visualisasi dari data skala abu-abu tersebut ke dalam lingkungan plot menggunakan perintah `plt.imshow` dengan peta warna kustom `cmap="gray"` untuk memunculkan representasi visualnya ke layar.



Memasuki proses persiapan data matriks, langkah awal dilakukan dengan menggandakan data skala abu-abu asli menggunakan perintah `citra_m.copy()`, yang langsung dikonversi menjadi tipe data desimal melalui fungsi `.astype(float)` lalu ditampung ke dalam variabel `copyCitra1`. Selanjutnya, informasi ukuran dari dokumen visual tersebut diuraikan ke dalam variabel baris `m1` dan variabel kolom `n1` memanfaatkan fitur `.shape`, di mana kedua nilai dimensi ini langsung dipakai untuk memesan ruang matriks kosong baru bernama `output1` dengan perintah `np.empty([m1, n1])`. Rangkaian instruksi ini diakhiri dengan melakukan validasi ukuran objek lewat perintah `print`, yang menampilkan penegasan bahwa salinan data serta matriks penampung keluaran telah memiliki resolusi spasial yang seragam yaitu $(2160, 3840)$, dengan detail nilai tinggi `m1` sebesar 2160 dan lebar `n1` sebesar 3840.



Langkah berikutnya masuk ke eksekusi algoritma konvolusi manual untuk pengolahan ketetangaan piksel yang tertera pada dokumen kode terakhir. Prosesnya berjalan lewat perulangan bersarang memakai dua perintah `for` untuk menyisir koordinat baris dari indeks 0 sampai `m1-1` dan bagian kolom dari indeks 0 sampai `n1-1`. Di setiap titik lokasi piksel pusat yang ditampung dalam variabel `a1` dan `b1`, sistem akan menghitung nilai lokal dengan menjumlahkan intensitas dari sembilan piksel bertetangga yaitu koordinat piksel itu sendiri ditambah delapan piksel lain yang mengelilinginya tepat dalam radius satu piksel. Hasil akumulasi total pada variabel `jumlah` tersebut kemudian dikalikan dengan sepersembilan ($1/9$) untuk mendapatkan nilai rata-rata spasialnya, lalu dimasukkan ke dalam matriks `output1` sebelum seluruh tipe data penampung akhir tersebut dikembalikan ke format standar citra integer 8-bit tanpa tanda (`np.uint8`) menggunakan fungsi `np.asarray`. Sebagai penutup, lingkungan visualisasi disiapkan melalui perintah `plt.subplots` dengan konfigurasi satu baris dua kolom berdimensi 10×10 yang diratakan posisinya lewat `axs.ravel()`, sehingga data awal skala abu-abu nampak di sisi kiri pada indeks `ax[0]` dengan judul 'Input Citra 1' dan hasil akhir pemrosesan filter rata-rata manual tampil di sisi kanan pada indeks `ax[1]` dengan judul 'Input Citra 2' memakai peta warna kustom `cmap="gray"`.

```

Di dalam [9]: #MohamadIrsyadIbrahimFaqih_202431179

citra_m2 = citra_m
img_median = citra_m2.copy()
img_median_after = cv2.medianBlur(img_median, 5)

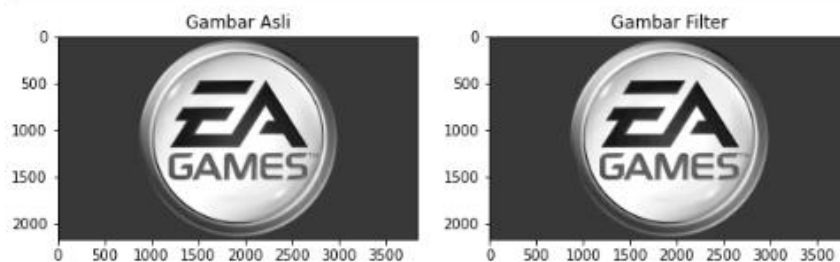
fig, axs = plt.subplots(1,2, figsize=(10,10))
ax = axs.ravel()

ax[0].imshow(img_median, cmap="gray")
ax[0].set_title("Gambar Asli")

ax[1].imshow(img_median_after, cmap="gray")
ax[1].set_title("Gambar Filter")

plt.show()

```



Langkah selanjutnya, kita beralih ke penerapan filter median otomatis yang alur kerja dimulai dengan memasukkan data `citra_m` ke dalam variabel baru `citra_m2`, yang kemudian langsung digandakan menggunakan perintah `.copy()` ke dalam variabel `img_median`. Langkah selanjutnya, operasi penyaringan dilakukan dengan memanfaatkan fungsi bawaan OpenCV yaitu `cv2.medianBlur(img_median, 5)` dengan ukuran window sebesar 5 untuk menghaluskan intensitas piksel bertetangga, lalu hasilnya disimpan ke dalam variabel `img_median_after`. Terakhir, sebuah kanvas visualisasi disiapkan lewat fungsi `plt.subplots` dengan format satu baris dua kolom berdimensi 10x10 dan diratakan posisinya menggunakan perintah `axs.ravel()`. Melalui pengaturan tersebut, data awal sebelum difilter ditampilkan pada indeks kiri `ax[0]` dengan judul "Gambar Asli", sementara data hasil pemrosesan filter median nampak di sisi kanan pada indeks `ax[1]` dengan judul "Gambar Filter" menggunakan konfigurasi peta warna abu-abu (`cmap="gray"`).

Di dalam [11]: #MohamadIrsyadIbrahimFaqih_202431179

```
citra_m3 = citra_m
copyCitra3 = citra_m3.copy().astype(float)

m1, n1 = copyCitra3.shape
output3 = np.empty([m1-1, n1-1])

for baris in range(1, m1-1):
    for kolom in range(1, n1-1):
        a1 = baris
        b1 = kolom
        arr = np.array([copyCitra3[a1-1, b1-1], copyCitra3[a1-1, b1], copyCitra3[a1-1, b1+1],
                        copyCitra3[a1, b1-1], copyCitra3[a1, b1+1],
                        copyCitra3[a1+1, b1-1], copyCitra3[a1+1, b1], copyCitra3[a1+1, b1+1]])

        minPiksel = np.amin(arr)
        maxPiksel = np.amax(arr)

        if (copyCitra3[baris, kolom] < minPiksel):
            output3[baris, kolom] = minPiksel
        else:
            if (copyCitra3[baris, kolom] > maxPiksel):
                output3[baris, kolom] = maxPiksel
            else:
                output3[baris, kolom] = copyCitra3[baris, kolom]

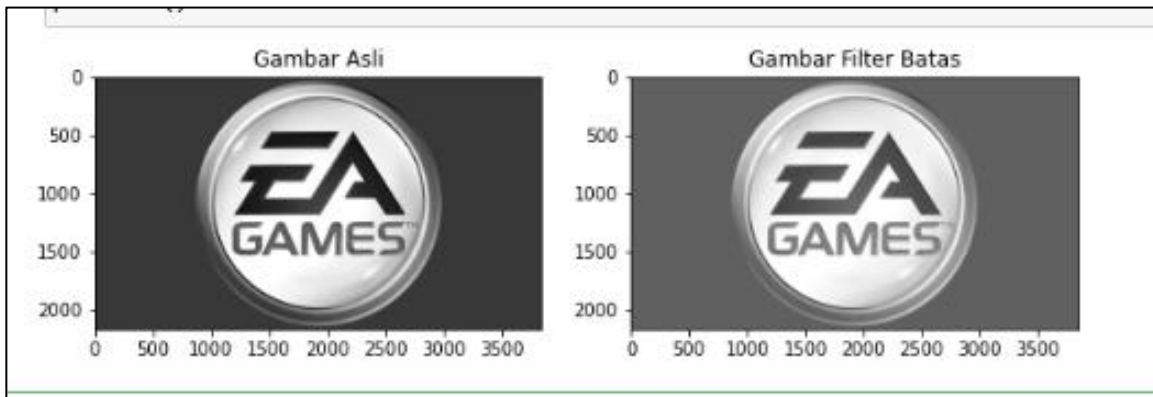
output3 = np.asarray(output3, dtype=np.uint8)

fig, axs = plt.subplots(1, 2, figsize = (10, 10))
ax = axs.ravel()

ax[0].imshow(citra_m3, cmap="gray")
ax[0].set_title("Gambar Asli")

ax[1].imshow(output3, cmap="gray")
ax[1].set_title("Gambar Filter Batas")

plt.show()
```



Langkah selanjutnya, kita beralih ke penerapan filter batas secara manual, Proses ini dimulai dengan menyalin matriks asal ke variabel `copyCitra3` dan mengubah tipenya menjadi float, lalu disiapkan matriks kosong `output3` berukuran `[m1-1, n1-1]` berdasarkan dimensi asli gambar. Pemrosesan piksel dilakukan menggunakan dua perulangan `for` untuk memeriksa area ketetanggaan, di mana delapan nilai piksel di sekitar posisi pusat dikumpulkan ke dalam array `arr` guna mencari nilai terendah melalui fungsi `np.amin(arr)` dan nilai tertinggi dengan `np.amax(arr)`. Penentuan nilai baru menggunakan logika pengkondisian: jika nilai piksel asli berada di bawah batas minimum ketetanggaannya maka diganti dengan nilai `minPiksel`, jika melebihi batas maksimum diganti dengan `maxPiksel`, dan jika berada di dalam rentang tersebut nilainya tetap dipertahankan. Setelah semua koordinat selesai diproses, tipe data matriks `output3` dikonversi kembali ke format standar gambar 8-bit (`np.uint8`) lewat perintah `np.asarray`, lalu hasilnya ditampilkan berdampingan menggunakan `plt.subplots` satu baris dua kolom untuk membandingkan intensitas gambar asli pada sisi kiri dan efek dari filter batas pada sisi kanan.

BAB IV

PENUTUP

Kesimpulan:

Berdasarkan serangkaian eksperimen yang telah dilaksanakan pada praktikum Operasi Konvolusi dan Metode Ketetanggaan Piksel, dapat disimpulkan bahwa kedua metode ini memiliki peran yang sangat krusial dalam manipulasi karakteristik spasial citra digital. Operasi konvolusi menggunakan kernel terbukti efektif untuk melakukan transformasi tingkat tinggi seperti penajaman tepi objek menggunakan kernel filter batas maupun penghalusan gambar menggunakan filter linear. Dalam hal penanganan kerusakan citra, pengujian reduksi derau menunjukkan bahwa Median Filter yang bersifat non-linear memiliki performa paling unggul dalam melenyapkan derau jenis Salt and Pepper tanpa merusak kontur asli dari objek gambar. Di sisi lain, Average Filter memberikan hasil pembersihan yang lebih menyeluruh pada derau jenis Gaussian dan Speckle, meskipun memberikan konsekuensi berupa efek pengaburan pada garis tepi objek akibat proses perataan nilai intensitas dengan piksel tetangga di sekitarnya. Sementara itu, implementasi algoritma Filter Batas secara manual membuktikan bahwa pembatasan nilai intensitas berdasarkan nilai minimum dan maksimum dari matriks ketetanggaan 3x3 mampu menstabilkan kontras lokal citra secara presisi dan terstruktur.

DAFTAR PUSTAKA

Dijaya, R. (2023). *Buku Ajar Pengolahan Citra Digital*. Umsida Press. <https://doi.org/10.21070/2023/978-623-464-075-5>

Fatkurrohman, W. (2024). Analisis efektivitas median filter pada peningkatan kualitas citra hasil scan dengan metrik MSE dan PSNR. *Jurnal Nusantara of Engineering (NOE)*, 5(1), 12–25. <https://doi.org/10.29407/noe.v5i1.21852>

Firmansyah, M. N. F. (2023). *Implementasi Connected Component Labeling memanfaatkan teori connectivity piksel pada pengolahan citra biner* [Skripsi, Institut Bisnis dan Informatika Stikom Surabaya]. Repositori Institusi Stikom Surabaya. <http://repository.dinamika.ac.id/id/eprint/7142/>

Ningrum, I. P., Putro, A. E., & Nursantika, D. (2022). Penapisan derau Gaussian, Speckle dan Salt&Pepper pada citra warna menggunakan metode penapisan spasial. *Jurnal Ilmiah Komputer Visi dan Pengolahan Citra*, 8(2), 89–98. <https://journal.universitaskebangsaan.ac.id/index.php/jiko/article/view/1120>

Putra, E. (2023). Klasifikasi citra menggunakan Convolutional Neural Network (CNN) dengan pendekatan ekstraksi fitur berbasis konvolusi. *Jurnal Teknologi Informasi dan Rekayasa Sistem*, 4(1), 45–54. <https://jurnal.unmer.ac.id/index.php/jtirs/article/view/9830>